



ÁREA TEMÁTICA: (marque uma das opções)

- ☐ Ensino de Matemática
☒ Matemática
☐ Probabilidade e Estatística

Mônadas: Uma Exposição Matemático-Funcional

Arthur P. de Aguiar Pereira¹

UEPG: Universidade Estadual de Ponta Grossa
25003009@uepg.br

Resumo: Este trabalho expõe formalmente as mônadas (*monads*) da programação funcional e sua fundamentação pela Teoria das Categorias, a partir da distinção entre valores e computações de Eugenio Moggi, sistematizada por Philip Wadler. Partindo da extensão de funções parciais, define-se a estrutura monádica por sua unidade e multiplicação e demonstra-se o resultado central: as leis do *bind* decorrem das leis de coerência com a naturalidade e equivalem a que a composição de Kleisli seja uma composição categorial, com identidades dadas pela unidade. As mônadas de exceção e de estado são construídas e verificadas, e sua composição é analisada por meio de leis distributivas: a mônada de exceção distribui sobre qualquer mônada, mas demonstra-se que, havendo ao menos dois estados e algum valor de erro, a mônada de estado não admite lei distributiva sobre a de exceção, um resultado de impossibilidade.

Palavras-chave: Mônadas, Programação Funcional, Teoria das Categorias, Categoria de Kleisli, Leis Distributivas.

Introdução: O Problema da Pureza e da Modularidade

Em linguagens puramente funcionais, as expressões denotam valores e gozam de transparência referencial, o que sustenta o raciocínio equacional e viabiliza a avaliação preguiçosa. Com efeitos como estado mutável ou exceções, porém, a avaliação deixa de ser puramente equacional e a ordem de avaliação pode importar [7]; representá-los explicitamente (recebendo e devolvendo o estado em cada função, por exemplo) sobrecarrega cada etapa e dificulta a composição dos programas.

Moggi [5] propôs representar essas noções de computação por *mônadas* (*monads*), distinguindo o valor produzido da computação que o produz, cuja natureza depende do efeito, e Wadler [7] levou-as à programação funcional. Na Seção 1, introduzimos a mônada, o *bind* e as leis monádicas; na Seção 2, provamos a correspondência entre as formulações operacional e categorial, construímos as mônadas de exceção e de estado e analisamos sua composição por leis distributivas.

1 Fundamentos: Mônadas como Noções de Computação

Antes da definição geral, um exemplo familiar. Uma função parcial $A \rightarrow B$ pode ser totalizada como $f : A \rightarrow B + E$, em que $B + E$ é o coproduto em **Set** (a união disjunta), com injecções ι_1, ι_2 , e $E = \{\perp\}$ representa a ausência de resultado. Pondo $\text{inv}(x) = \iota_1(1/x)$ para $x \neq 0$ e $\iota_2(\perp)$ para $x = 0$, e $\text{raiz}(x) = \iota_1(\sqrt{x})$ para $x \geq 0$ e $\iota_2(\perp)$ para $x < 0$, obtemos funções *totais* $\mathbb{R} \rightarrow \mathbb{R} + E$, que não se compõem: *raiz* espera um número, e *inv* devolve um elemento de $\mathbb{R} + E$. *Estende-se* então *raiz* a *raiz** : $\mathbb{R} + E \rightarrow \mathbb{R} + E$, com $\text{raiz}^*(\iota_1(y)) = \text{raiz}(y)$ e $\text{raiz}^*(\iota_2(\perp)) = \iota_2(\perp)$, e compõe-se: $\text{raiz}^* \circ \text{inv}$ é a função parcial

¹Aluno de graduação em Matemática Aplicada.

$x \mapsto \sqrt{1/x}$, definida exatamente para $x > 0$. Duas observações organizam o que segue: todo valor se lê como computação sem erro, por $\eta(x) = \iota_1(x)$, neutra para essa composição; e a extensão $(-)^*$ repete, para cada f , o mesmo gesto de propagar o erro e prosseguir no sucesso. É esse gesto que a mônada isola: $B + E$ é um caso particular da forma geral TB , com $(-)^* + E$ substituído por um endofuntor arbitrário.

Seguindo Moggi [5], seja $\mathcal{C}$ uma categoria cujos objetos representam tipos; uma computação que produz valores de tipo A é um objeto TA , com T um endofuntor de $\mathcal{C}$, cujo significado depende da mônada e não se reduz a “um valor mais um contexto”: na de exceção, um elemento de TA é um sucesso ou um erro; na de estado, uma computação que transforma estados (Seção 2.2). “Contexto computacional” é só um nome genérico. Escrevemos $T^2 := T \circ T$, $T^3 := T \circ T^2$ e $A \in \mathcal{C}$ para “ A é objeto de $\mathcal{C}$ ”; os enunciados gerais são livres de elementos (*point-free*), e a notação $x \in A$, $f(x)$ indica que estamos em **Set**.

1.1 A Estrutura Monádica

Definição 1.1 (Mônada). Uma *mônada* sobre $\mathcal{C}$ é uma tripla (T, η, μ) formada por um endofuntor $T : \mathcal{C} \rightarrow \mathcal{C}$ e transformações naturais $\eta : \text{Id}_{\mathcal{C}} \Rightarrow T$ (*unidade*) e $\mu : T^2 \Rightarrow T$ (*multiplicação*) tais que, para todo $A \in \mathcal{C}$ [3, 5],

$$\mu_A \circ \mu_{TA} = \mu_A \circ T(\mu_A), \quad \mu_A \circ \eta_{TA} = \text{id}_{TA} = \mu_A \circ T(\eta_A).$$

A unidade $\eta_A : A \rightarrow TA$ insere um valor no contexto sem produzir efeito algum; a multiplicação $\mu_A : T^2A \rightarrow TA$ funde duas camadas de contexto em uma, especificando como se acumulam os efeitos de duas etapas consecutivas. É nessa escolha que reside o conteúdo semântico de cada mônada. A naturalidade significa que, para todo $f : A \rightarrow B$, $T(f) \circ \eta_A = \eta_B \circ f$ e $\mu_B \circ T^2(f) = T(f) \circ \mu_A$, isto é, η e μ são compatíveis com os morfismos da categoria: inserir e depois transformar sob T equivale a transformar e depois inserir, e fundir camadas comuta com aplicar f dentro delas. Essa exigência não é dispensável: é ela que permite deslocar η e μ através de $T(f)$ nas demonstrações da Seção 2.1, e por isso as leis do *bind* decorrem da estrutura em vez de serem axiomas independentes.

1.2 Extensão de Kleisli, Bind e Leis Monádicas

Definição 1.2 (Extensão e composição de Kleisli). A *extensão de Kleisli* de um morfismo $f : A \rightarrow TB$ é $f^* := \mu_B \circ T(f) : TA \rightarrow TB$, e a *composição de Kleisli* de f com $g : B \rightarrow TC$ é $g \circ_T f := g^* \circ f = \mu_C \circ T(g) \circ f$.

$$\begin{array}{ccccccc} A & \xrightarrow{f} & TB & \xrightarrow{T(g)} & T^2C & \xrightarrow{\mu_C} & TC \\ & \searrow & & & \searrow & & \\ & & & & g \circ_T f & & \end{array}$$

O morfismo g , que espera um valor de tipo B , não se aplica a TB ; aplica-se $T(g)$, que age sob o contexto e devolve duas camadas, fundidas por μ_C . É o gesto do exemplo de abertura, agora em qualquer mônada.

Na programação funcional, a extensão aparece como o encadeamento *bind* [7]: em **Set**, para $m \in TA$, $m \gg f := f^*(m) = \mu_B(T(f)(m)) \in TB$, isto é, $T(f)$ gera duas camadas, que μ_B funde. O *bind* é a notação operacional de $(-)^*$, e $g \circ_T f = \lambda x. f(x) \gg g$. Suas três leis [5, 7] são as *leis de identidade e associatividade da composição de Kleisli* $\circ_T$, com identidades η_A ; enunciemo-las em forma livre de

elementos, para $f : A \rightarrow TB$ e $g : B \rightarrow TC$, e em **Set**, para $x \in A$ e $m \in TA$:

$$f^* \circ \eta_A = f, \quad \eta_A(x) \gg f = f(x), \quad (1)$$

$$(\eta_A)^* = \text{id}_{TA}, \quad m \gg \eta_A = m, \quad (2)$$

$$(g^* \circ f)^* = g^* \circ f^*, \quad (m \gg f) \gg g = m \gg (\lambda x. f(x) \gg g). \quad (3)$$

Assim, por (1) e (2), η não introduz efeito e é neutra para $\circ_T$, e, por (3), o resultado não depende do agrupamento das etapas. O *bind* não é, portanto, uma operação *ad hoc*: suas leis traduzem uma estrutura categorial, o que a Proposição 2.2 torna preciso. (Variáveis livres, como na notação *do*, exigem ainda uma força $A \times TB \rightarrow T(A \times B)$ [5].)

2 Formalização Categorial: Coerência, Correspondência e Exemplos

Em forma diagramática, as leis da Definição 1.1 afirmam que comutam os diagramas

$$\begin{array}{ccc} T^3A & \xrightarrow{\mu_{TA}} & T^2A \\ T(\mu_A) \downarrow & & \downarrow \mu_A \\ T^2A & \xrightarrow{\mu_A} & TA \end{array} \quad \begin{array}{ccccc} TA & \xrightarrow{\eta_{TA}} & T^2A & \xleftarrow{T(\eta_A)} & TA \\ & \searrow \text{id}_{TA} & \downarrow \mu_A & & \swarrow \text{id}_{TA} \\ & & TA & & \end{array}$$

isto é, a fusão de três camadas não depende do agrupamento, e uma camada acrescentada “por fora” (η_{TA}) ou “por dentro” ($T(\eta_A)$) é desfeita por μ_A .

2.1 A Correspondência entre as Formulações Operacional e Categorial

Proposição 2.1. Se (T, η, μ) é uma mônada sobre $\mathcal{C}$, (1) a (3) valem para quaisquer $f : A \rightarrow TB$ e $g : B \rightarrow TC$.

Demonstração. Cada lei usa uma parte da Definição 1.1: (1), a naturalidade de η e a unidade externa; (2), a unidade interna; (3), a functorialidade de T , a associatividade de μ e a naturalidade de μ . Em (1), $f^* \circ \eta_A = \mu_B \circ T(f) \circ \eta_A = \mu_B \circ \eta_{TB} \circ f = f$. A igualdade (2) é a lei de unidade interna: $(\eta_A)^* = \mu_A \circ T(\eta_A) = \text{id}_{TA}$. Em (3), a naturalidade de μ aplicada a g fornece $\mu_{TC} \circ T^2(g) = T(g) \circ \mu_B$; as três primeiras igualdades abaixo usam, nessa ordem, functorialidade, associatividade e naturalidade:

$$\begin{aligned} (g^* \circ f)^* &= \mu_C \circ T(\mu_C) \circ T^2(g) \circ T(f) = \mu_C \circ \mu_{TC} \circ T^2(g) \circ T(f) \\ &= \mu_C \circ T(g) \circ \mu_B \circ T(f) = g^* \circ f^*. \end{aligned} \quad \square$$

Proposição 2.2 (*Bind e composição de Kleisli*). Suponha dados, para cada $A \in \mathcal{C}$, um objeto TA e um morfismo $\eta_A : A \rightarrow TA$ e, para cada $f : A \rightarrow TB$, um morfismo $f^* : TA \rightarrow TB$, e defina $g \circ_T f := g^* \circ f$. Então (1) a (3) valem se, e somente se, $\mathcal{C}_T$, com os objetos de $\mathcal{C}$, $\text{Hom}_{\mathcal{C}_T}(A, B) := \text{Hom}_{\mathcal{C}}(A, TB)$, composição $\circ_T$ e identidades η_A , é uma categoria (isto é, $\circ_T$ é composição categorial com identidades dadas pela unidade). Em particular, para toda mônada, $\mathcal{C}_T$ é uma categoria, a categoria de Kleisli.

Demonstração. Sejam $f : A \rightarrow TB$, $g : B \rightarrow TC$ e $h : C \rightarrow TD$. Se as leis valem, $f \circ_T \eta_A = f^* \circ \eta_A = f$, $\eta_B \circ_T f = (\eta_B)^* \circ f = f$ e $h \circ_T (g \circ_T f) = h^* \circ g^* \circ f = (h^* \circ g)^* \circ f = (h \circ_T g) \circ_T f$. Reciprocamente, se $\mathcal{C}_T$ é uma categoria, $f \circ_T \eta_A = f$ é (1); a lei $\eta_B \circ_T f = f$ aplicada a $f := \text{id}_{TB} \in \text{Hom}_{\mathcal{C}_T}(TB, B)$ dá (2); e a associatividade aplicada a $\text{id}_{TA} \in \text{Hom}_{\mathcal{C}_T}(TA, A)$, f e g dá $g^* \circ f^* = g \circ_T (f \circ_T \text{id}_{TA}) = (g \circ_T f) \circ_T \text{id}_{TA} = (g^* \circ f)^*$, que é (3). A última afirmação segue da Proposição 2.1. $\square$

Programar com *bind* é, assim, compor morfismos em $\mathcal{C}_T$. A Proposição 2.2 não pressupõe uma mônada, e de fato uma *tripla de Kleisli* $(T, \eta, (-)^*)$, com T dado só nos objetos e satisfazendo (1) a (3), basta para *reconstruí-la*, com $T(f) := (\eta_B \circ f)^*$ para $f : A \rightarrow B$ e $\mu_A := (\text{id}_{TA})^*$, e as leis fornecem a functorialidade, a naturalidade e a coerência. As construções são mutuamente inversas [4, 5]: mônada e tripla de Kleisli apresentam a mesma estrutura.

2.2 As Mônadas de Exceção e de Estado

Seja $\mathcal{C}$ uma categoria com coprodutos binários e $E \in \mathcal{C}$ um objeto de erros. Denotamos por ι_1, ι_2 as injeções de um coproduto, por $[u, v]$ o copareamento e, para $u : X \rightarrow X'$ e $v : Y \rightarrow Y'$, por $u + v := [\iota_1 \circ u, \iota_2 \circ v]$, de modo que $(u + v) \circ \iota_1 = \iota_1 \circ u$, $(u + v) \circ \iota_2 = \iota_2 \circ v$ e $+$ é functorial. Abaixo, ι_1, ι_2 são as injeções de $A + E$ e $\bar{\iota}_1, \bar{\iota}_2$ as de $(A + E) + E$. A *mônada de exceção* é dada por $TA = A + E$, $T(f) = f + \text{id}_E$, $\eta_A = \iota_1$ e $\mu_A = [\text{id}_{A+E}, \iota_2] : (A + E) + E \rightarrow A + E$. Assim, μ_A funde dois níveis de sucesso e preserva um erro de qualquer nível, e f^* recupera a extensão da Seção 1; $E = 1$ terminal dá a mônada *Maybe*.

Proposição 2.3. *A tripla (T, η, μ) acima é uma mônada sobre $\mathcal{C}$.*

Demonstração. A functorialidade de T e a naturalidade de η decorrem das identidades sobre $u + v$. As demais igualdades têm domínio coproduto, e $u, v : X + Y \rightarrow Z$ são iguais se, e somente se, $u \circ \iota_1 = v \circ \iota_1$ e $u \circ \iota_2 = v \circ \iota_2$ (unicidade na propriedade universal do coproduto). Detalhamos a unidade interna; as outras são análogas. Sobre ι_1 , tem-se $\mu_A \circ T(\eta_A) \circ \iota_1 = \mu_A \circ \bar{\iota}_1 \circ \eta_A = \eta_A = \iota_1$; sobre ι_2 , tem-se $\mu_A \circ T(\eta_A) \circ \iota_2 = \mu_A \circ \bar{\iota}_2 = \iota_2$. Logo $\mu_A \circ T(\eta_A) = \text{id}_{A+E}$. $\square$

Em **Set**, fixado um conjunto S de estados, a *mônada de estado* tem $TA = (A \times S)^S$: uma computação $h : S \rightarrow A \times S$ recebe um estado e devolve um valor e um novo estado. Definem-se $T(f)(h) = (f \times \text{id}_S) \circ h$, $\eta_A(x) = \lambda s. (x, s)$ e $\mu_A(F)(s) = g(s')$ se $F(s) = (g, s')$, em que $F \in T^2A = ((A \times S)^S \times S)^S$ produz, a partir de s , uma computação g e um estado intermediário s' . Esta construção usa produtos e exponenciais, vale em qualquer categoria cartesiana fechada e provém da adjunção $(- \times S) \dashv (-)^S$ [3].

Proposição 2.4. *A tripla (T, η, μ) acima é uma mônada sobre **Set**.*

Demonstração. A functorialidade de T e a naturalidade de η são imediatas, e a naturalidade de μ segue de $T^2(f)(F)(s) = (T(f)(g), s')$ quando $F(s) = (g, s')$. Nas unidades, $\mu_A(\eta_{TA}(h))(s) = h(s)$ e, escrevendo $h(s) = (a, s_1)$, $\mu_A(T(\eta_A)(h))(s) = \eta_A(a)(s_1) = h(s)$. Na associatividade, se $G(s) = (F, s_1)$ e $F(s_1) = (h, s_2)$, ambos os lados valem $h(s_2)$ em s . $\square$

Um caso concreto: com $S = \mathbb{N}$, a computação $\text{prox} \in T\mathbb{N}$, $\text{prox}(n) = (n, n + 1)$, devolve o contador atual e o incrementa. Encadeada consigo mesma a partir de $s = 0$, a primeira etapa produz o valor 0 e o estado 1, e a segunda, 1 e o estado 2: o contador avança sem que nenhuma etapa o mencione. É essa passagem implícita que μ_A codifica: em $\mu_A(F)(s)$, o estado intermediário s' devolvido pela primeira etapa alimenta a segunda. Rotular elementos com índices sucessivos é esse encadeamento, repetido.

2.3 Limitações da Composição entre Mônadas

Um programa realista raramente usa um efeito só (quer manter estado e poder falhar), o que leva a perguntar se duas mônadas se combinam numa terceira. Dadas (T, η^T, μ^T) e (U, η^U, μ^U) sobre $\mathcal{C}$, compor os endofuntores é imediato, mas $T \circ U$ não herda, em geral, estrutura monádica: para fundir $TUTU$ em TU é preciso reorganizar o par intercalado UT em TU , e em geral não há transformação natural $UT \Rightarrow TU$ que o faça de modo compatível com as unidades e as multiplicações das duas mônadas. Uma tal transformação é uma *lei distributiva* [1]. Escrevemos $(\alpha H)_A := \alpha_{HA}$ e $(H\alpha)_A := H(\alpha_A)$. Abaixo,

(i) e (ii) exprimem a compatibilidade com as unidades (permutar uma camada recém-inserida equivale a inseri-la já do outro lado), e (iii) e (iv), com as multiplicações (fundir duas camadas e depois permutar equivale a permutar cada uma e depois fundir).

Definição 2.5 (Lei distributiva). Uma *lei distributiva* de U sobre T é uma transformação natural $\lambda : U \circ T \Rightarrow T \circ U$ tal que

$$\begin{aligned} \text{(i)} \quad \lambda \circ \eta^U T &= T \eta^U : T \Rightarrow TU, & \text{(iii)} \quad \lambda \circ \mu^U T &= T \mu^U \circ \lambda U \circ U \lambda : U^2 T \Rightarrow TU, \\ \text{(ii)} \quad \lambda \circ U \eta^T &= \eta^T U : U \Rightarrow TU, & \text{(iv)} \quad \lambda \circ U \mu^T &= \mu^T U \circ T \lambda \circ \lambda T : UT^2 \Rightarrow TU. \end{aligned}$$

Dada λ , $T \circ U$ é uma mônada [1], com unidade $\eta_{UA}^T \circ \eta_A^U$ e a multiplicação

$$TUTUA \xrightarrow{T(\lambda_{UA})} TTUUA \xrightarrow{\mu_{U^2A}^T} TUUA \xrightarrow{T(\mu_A^U)} TUA,$$

em que λ troca o par intercalado UT por TU , e cada mônada funde então as suas camadas. Uma lei distributiva de T sobre U , $T \circ U \Rightarrow U \circ T$, satisfaz (i) a (iv) com T e U permutados e torna $U \circ T$ uma mônada. A seguir, U é a mônada de exceção, $UA = A + E$.

Proposição 2.6 (A exceção distribui sobre qualquer mônada). Para toda mônada T sobre $\mathcal{C}$, $\lambda_A := [T(\iota_1), \eta_{A+E}^T] : TA + E \rightarrow T(A + E)$ é uma lei distributiva de U sobre T ; logo $A \mapsto T(A + E)$ é uma mônada.

Esboço. Como na Proposição 2.3, compara-se sobre as injeções: (i) é a definição de λ ; nas de sucesso, o resto segue da naturalidade de η^T e μ^T e das leis de U ; nas de erro, ambos os lados valem $\eta_{A+E}^T \circ \iota_2$. $\square$

A exceção é compatível com uma mônada arbitrária porque λ só usa o que toda mônada possui, independentemente da estrutura específica de T : no sucesso, $T(\iota_1)$; no erro, $\eta^T \circ \iota_2$, que introduz o erro em T sem efeito algum. Com T de estado, obtém-se $A \mapsto ((A + E) \times S)^S$, em que o estado *sobrevive* à falha. A ordem inversa, $(A \times S)^S + E$, não se obtém assim, por uma razão semântica: o erro está *fora* da função, e uma computação que falha não reporta estado final, embora falhar dependa do estado lido. O resultado seguinte não é uma construção, mas uma impossibilidade, sob hipóteses explícitas sobre S e E .

Proposição 2.7 (Impossibilidade). Sejam T a mônada de estado sobre **Set**, com S tendo ao menos dois elementos, e U a mônada de exceção sobre **Set**, com $E \neq \emptyset$. Não existe lei distributiva de T sobre U .

Demonstração. A ideia é que uma computação que só produz erros tem de atravessar λ como erro; confrontando isso com (iii') em dois casos opostos, os dois lados discordam. Suponha, pois, que $\lambda : T \circ U \Rightarrow U \circ T$ seja uma tal lei. Das condições (i) a (iv) com T e U permutados, usaremos apenas (ii') $\lambda_A \circ T(\eta_A^U) = \eta_{TA}^U$ e (iii') $\lambda_A \circ \mu_{UA}^T = U(\mu_A^T) \circ \lambda_{TA} \circ T(\lambda_A)$. Dizemos que $k \in TUA$ só produz erros se $k(s) \in \iota_2(E) \times S$ para todo s .

Passo 1: se $k \in TUA$ só produz erros, $\lambda_A(k)$ é um erro. Se $k(s) = (\iota_2(e_s), t_s)$, então $k = TU(!)(k_0)$, com $! : \emptyset \rightarrow A$ e $k_0(s) := (\iota_2(e_s), t_s) \in U\emptyset \times S$. Por naturalidade, $\lambda_A(k) = UT(!)(\lambda_\emptyset(k_0))$. Como $S \neq \emptyset$, $T\emptyset = \emptyset$ e $UT\emptyset = \emptyset + E$; logo $\lambda_\emptyset(k_0) = \iota_2(e')$ para algum $e' \in E$, e $\lambda_A(k) = \iota_2(e')$.

Passo 2: os dois casos possíveis para $\lambda_A(w)$ contradizem (iii'). Fixe $A = \{*\}$, $s_0 \neq s_1$ em S , $e \in E$ e $w \in TUA$ dada por $w(s_0) = (\iota_1(*), s_0)$ e $w(s) = (\iota_2(e), s)$ para $s \neq s_0$: w só tem sucesso ao ler s_0 .

Caso $\lambda_A(w) = \iota_2(e')$. Tome $z := \lambda s. (w, s_0) \in T^2UA$. Então $\mu_{UA}^T(z) = \lambda s. (\iota_1(*), s_0) = T(\eta_A^U)(c)$, com $c := \lambda s. (*, s_0)$, e por (ii') o lado esquerdo de (iii') é o sucesso $\iota_1(c)$; mas $T(\lambda_A)(z) = \lambda s. (\iota_2(e'), s_0)$ só produz erros, e pelo Passo 1 (no objeto TA) λ_{TA} o leva num erro, que $U(\mu_A^T) = \mu_A^T + \text{id}_E$ preserva.

Caso $\lambda_A(w) = \iota_1(h)$. Tome $z := \lambda s. (w, s_1)$. Então $\mu_{UA}^T(z) = \lambda s. (\iota_2(e), s_1)$ só produz erros, e pelo Passo 1 o lado esquerdo de (iii') é um erro; mas $T(\lambda_A)(z) = \lambda s. (\iota_1(h), s_1) = T(\eta_{TA}^U)(h')$, com $h' := \lambda s. (h, s_1) \in T^2A$, e por (ii') o lado direito é o sucesso $\iota_1(\mu_A^T(h'))$. Em ambos os casos, (iii') falha. $\square$



Essas dificuldades motivam os *monad transformers* [2] e os efeitos algébricos [6]: combinar efeitos é uma escolha de projeto, não uma operação automática.

Conclusão

Este trabalho é predominantemente expositivo e de formalização: não propõe uma nova teoria de mônadas, mas integra numa só exposição a definição categorial, a leitura operacional do *bind*, a categoria de Kleisli, as mônadas de exceção e de estado e as leis distributivas, com demonstrações explícitas.

O resultado central é que as leis do *bind* decorrem das leis de coerência de η e μ com sua naturalidade (Proposição 2.1) e valem exatamente quando $\circ_T$ é uma composição categorial (Proposição 2.2): programar com $\gg=$ é compor morfismos de Kleisli, e a tripla de Kleisli apresenta a mesma estrutura. As mônadas de exceção e de estado (Proposições 2.3 e 2.4) mostram como a escolha de μ fixa o efeito. Na composição de efeitos, a ordem importa: a exceção distribui sobre qualquer mônada (Proposição 2.6), mas, com ao menos dois estados e algum erro, o estado *não* admite lei distributiva sobre a exceção (Proposição 2.7), um resultado de impossibilidade, e não apenas a ausência de uma construção.

Como desenvolvimento futuro, pretende-se estudar os *monad transformers* [2] como forma modular de combinar efeitos, contornando essa impossibilidade, e compará-los com os efeitos algébricos [6].

Nota sobre o uso de inteligência artificial

Na preparação deste trabalho utilizou-se o modelo de linguagem Claude Opus 5 (Anthropic) como auxílio à redação e à revisão do texto e das demonstrações. Todo o conteúdo matemático e as decisões de exposição são de responsabilidade do autor, que conferiu integralmente o material aqui apresentado.

Referências

- [1] BECK, J. Distributive laws. In *Seminar on Triples and Categorical Homology Theory*, B. Eckmann, Ed., vol. 80 of *Lecture Notes in Mathematics*. Springer, Berlin, Heidelberg, 1969, pp. 119–140.
- [2] LIANG, S., HUDAK, P., AND JONES, M. Monad transformers and modular interpreters. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '95)* (San Francisco, CA, 1995), ACM, pp. 333–343.
- [3] MAC LANE, S. *Categories for the Working Mathematician*, 2nd ed., vol. 5 of *Graduate Texts in Mathematics*. Springer, New York, NY, 1998.
- [4] MANES, E. G. *Algebraic Theories*, vol. 26 of *Graduate Texts in Mathematics*. Springer, New York, NY, 1976.
- [5] MOGGI, E. Notions of computation and monads. *Information and Computation* 93, 1 (1991), 55–92.
- [6] PLOTKIN, G. D., AND PRETNAR, M. Handling algebraic effects. *Logical Methods in Computer Science* 9, 4 (2013), 1–36.
- [7] WADLER, P. Monads for functional programming. In *Advanced Functional Programming: First International Spring School on Advanced Functional Programming Techniques*, J. Jeuring and E. Meijer, Eds., vol. 925 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 1995, pp. 24–52.